Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Data Structures & Algorithms

BCA — Semester 3 — 2019 — Answer Sheet

Subject Code: CACS201

Group A

Attempt all questions. [10×1=10]

1. Tick the correct answer for the following MCQs:

[10×1]

Answer: i) What is the measurement for time complexity of an algorithm?
Ans: c) Counting number of key operations
Time complexity is measured by counting the number of fundamental operations (comparisons, assignments, arithmetic operations) performed by an algorithm as a function of input size n. It describes how runtime grows with input size, typically expressed using Big-O notation.

ii) Result of postfix evaluation: 5 7 4 – * 8 4 / + ?
Ans: d) 17
Evaluation using stack:
• Push 5, 7, 4
• Encounter '-': pop 4, 7 → 7-4 = 3. Push 3
• Encounter '*': pop 3, 5 → 5*3 = 15. Push 15
• Push 8, 4
• Encounter '/': pop 4, 8 → 8/4 = 2. Push 2
• Encounter '+': pop 2, 15 → 15+2 = **17**

iii) Recursive formula for post order traversal of binary tree?
Ans: c) Left-Right-Root
Post-order traversal visits: left subtree → right subtree → root node. The recursive formula is: PostOrder(node) = PostOrder(node.left) + PostOrder(node.right) + Visit(node).

iv) Number of disk movements in TOH with 4 disks?
Ans: d) 15
Tower of Hanoi requires 2^n - 1 disk movements for n disks. For n = 4: 2^4 - 1 = 16 - 1 = **15 movements**.

v) Big-Oh of best case complexity of insertion sort?
Ans: a) O(n)
In the best case (already sorted array), insertion sort makes n-1 comparisons and 0 swaps. Each element is compared once with its predecessor, giving linear time complexity O(n).

vi) How does rear index incremented in circular queue?
Ans: b) rear=(rear+1)%SIZE
In a circular queue, the modulo operator wraps the rear pointer around to the beginning when it reaches the end of the array, enabling efficient reuse of vacant positions at the front.

vii) Variation of linked list where none of the node contains NULL pointer?
Ans: c) Circular
In a circular linked list, the last node's next pointer points back to the first node instead of NULL. Thus, no node contains a NULL pointer, creating a closed loop structure.

viii) Data structure used in DFS of graph?
Ans: a) Stack
Depth First Search (DFS) uses a stack (explicit or implicit via recursion) to explore vertices. It goes as deep as possible along each branch before backtracking.

ix) True for B-Tree of order M?
Ans: d) All non-leaf nodes with M-1 keys must have M number of children

In a B-Tree of order M, every non-leaf node with k keys must have exactly k+1 children. If a non-leaf node has M-1 keys, it must have M children. All leaves are at the same depth.

x) Which is not a hash function?
Ans: c) Chaining
Chaining is a collision resolution technique, not a hash function. Hash functions include division method, mid-square method, folding method, and multiplicative method. Chaining handles collisions by storing colliding elements in linked lists at each hash table index.

Group B

Attempt any SIX questions. [6×5=30]

2. What is Data Structure? Show the status of stack converting following infix expression to postfix P + Q – (R*S/T+U)-V*W

[1+4]

Answer: Data Structure:
A data structure is a specialized format for organizing, storing, and managing data so that it can be accessed and modified efficiently. It defines the relationship between data elements and the operations that can be performed on them. Examples include arrays, stacks, queues, linked lists, trees, and graphs. The choice of data structure affects program efficiency in terms of time and space complexity.

Infix to Postfix Conversion: P + Q – (R*S/T+U)-V*W
Operator precedence: * and / (higher), + and - (lower), parentheses override.
SymbolStackPostfix Expression PEmptyP ++P Q+P Q --P Q + (- (P Q + R- (P Q + R *- (*P Q + R S- (*P Q + R S /- (/P Q + R S * T- (/P Q + R S * T +- (+P Q + R S * T / U- (+P Q + R S * T / U)-P Q + R S * T / U + --P Q + R S * T / U + - V-P Q + R S * T / U + - V *- *P Q + R S * T / U + - V W- *P Q + R S * T / U + - V W EndEmptyP Q + R S * T / U + - V W * - **Final Postfix Expression:** P Q + R S * T / U + - V W * -

3. Write binary search. Consider hash table of size 10; insert keys 62, 37, 36, 44, 67, 91, 107 using linear probing.

[2.5+2.5]

Answer: Binary Search Algorithm:
Binary search is an efficient searching algorithm that works on sorted arrays by repeatedly dividing the search interval in half.
Algorithm BinarySearch(A, n, key): 1. Set low = 0, high = n - 1 2. While low <= high do: a. mid = (low + high) / 2 b. If A[mid] == key: Return mid // Element found c. Else If key < A[mid]: high = mid - 1 d. Else: low = mid + 1 3. Return -1 // Element not found **Time Complexity:** O(log n) for average and worst case, O(1) for best case.
Space Complexity: O(1) for iterative, O(log n) for recursive.
Requirement: Array must be sorted.

Hash Table Insertion using Linear Probing:
Hash function: h(key) = key % 10 (table size = 10)
Linear probing: if collision, try (h(key) + i) % 10 for i = 1, 2, 3...
Keyh(key) = key % 10PositionCollision? 6262 % 10 = 22No 3737 % 10 = 77No 3636 % 10 = 66No 4444 % 10 = 44No 6767 % 10 = 77 occupiedYes (7+1) % 10 = 88No 9191 % 10 = 11No 107107 % 10 = 77 occupiedYes (7+1) % 10 = 8 occupiedYes (7+2) % 10 = 99No **Final Hash Table: Index**0123456789 **Key**-9162-44-363767107

4. What are deterministic and non-deterministic algorithms? Explain greedy algorithm.

[2.5+2.5]

Answer: Deterministic Algorithm:
A deterministic algorithm is one that, given a particular input, will always produce the same output and follow the same sequence of steps. Every step has a uniquely defined next action. There is no element of randomness or choice in the execution path.
Characteristics:
• Same input always yields same output
• Execution path is predictable and repeatable
• Examples: Bubble sort, Binary search, Linear search

Non-Deterministic Algorithm:
A non-deterministic algorithm is one where the next state or output is not uniquely determined by the current state and input. It may have multiple possible execution paths, use randomness, or make arbitrary choices at certain steps.
Characteristics:
• Same input may produce different outputs on different runs
• May use random numbers or oracles to make choices
• Examples: Randomized quicksort, Monte Carlo algorithms, genetic algorithms, simulated annealing
• In theoretical computer science, non-deterministic algorithms are used to define the complexity class NP

Greedy Algorithm:
A greedy algorithm is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit (locally optimal choice) with the hope that these local optimizations will lead to a globally optimal solution.
Characteristics of Greedy Algorithm:
• Makes the locally optimal choice at each step
• Never reconsiders or reverses its choices (no backtracking)
• Simple and efficient when applicable
• Does not always produce the globally optimal solution
Components of Greedy Algorithm:
1. **Candidate set:** The set of potential choices
2. **Selection function:** Chooses the best candidate to add to the solution
3. **Feasibility function:** Checks if a candidate can contribute to the solution
4. **Objective function:** Assigns a value to a solution
5. **Solution function:** Indicates when a complete solution is found
Examples of Greedy Algorithms:
• **Dijkstra's algorithm:** Finds shortest path in a graph
• **Prim's and Kruskal's algorithms:** Find minimum spanning tree
• **Huffman coding:** Optimal prefix codes for data compression
• **Fractional knapsack:** Selects items with highest value-to-weight ratio
• **Activity selection:** Selects maximum number of non-overlapping activities

Example — Coin Change Problem (Greedy Approach):
To make change for 37 using denominations {25, 10, 5, 1}:
• Pick 25 (largest ≤ 37), remaining = 12
• Pick 10 (largest ≤ 12), remaining = 2
• Pick 1 (largest ≤ 2), remaining = 1
• Pick 1 (largest ≤ 1), remaining = 0

Total coins = 4 (optimal for US coin denominations)
Note: Greedy does NOT always give optimal solution for arbitrary denominations.

5. Draw a BST from the string DATASTRUCTURE and traverse in post order and pre-order.

[2+1.5+1.5]

Answer: Binary Search Tree (BST) Construction from "DATASTRUCTURE":
String: D A T A S T R U C T U R E
Ignoring duplicates (or handling them as per convention), unique characters in order of appearance: D, A, T, S, R, U, C, E
Insertion Process:
1. Insert D → D becomes root
2. Insert A → A < D, so A goes to left of D
3. Insert T → T > D, so T goes to right of D
4. Insert A → A already exists (skip or place in left subtree)
5. Insert S → S < T, S goes to left of T
6. Insert T → T already exists (skip)
7. Insert R → R < T, R < S, so R goes to left of S
8. Insert U → U > T, so U goes to right of T
9. Insert C → C < D, C > A, so C goes to right of A
10. Insert T → skip
11. Insert U → skip
12. Insert R → skip
13. Insert E → E < D, E > A, E < C, so E goes to left of C
Final BST Structure:
D / \ A T \ / \ C S U / / E R **Preorder Traversal (Root-Left-Right):** D A C E T S R U
Step by step:
• Visit D (root)
• Go left to A, visit A
• Go right to C, visit C
• Go left to E, visit E
• Back to C, go right (null)
• Back to A, go left (null)
• Back to D, go right to T, visit T
• Go left to S, visit S
• Go left to R, visit R
• Back to S, go right (null)
• Back to T, go right to U, visit U

Postorder Traversal (Left-Right-Root): E C A R S U T D
Step by step:
• From D, go left to A, go right to C, go left to E
• Visit E (no children)
• Visit C (right done)
• Visit A (right done)
• From D, go right to T, go left to S, go left to R
• Visit R (no children)
• Visit S (right done)
• Go right to U, visit U (no children)
• Visit T (both subtrees done)
• Visit D (root, both subtrees done)

Note: If all characters including duplicates are inserted (with duplicates going to right subtree), the tree would

be larger. The standard convention for BST with duplicates places them in the right subtree.

6. Define circular queue. How does it overcome the limitation of linear queue? Explain.

[1+4]

Answer: Circular Queue:
A circular queue is a linear data structure that follows the FIFO (First-In-First-Out) principle, but unlike a linear queue, the last position is connected back to the first position to form a circle. This allows efficient reuse of vacant positions created by dequeue operations at the front.

Limitations of Linear Queue:
In a linear queue implemented using an array, elements are inserted at the rear and deleted from the front. As dequeue operations occur, the front index moves forward, creating vacant spaces at the beginning of the array. However, the rear can only move forward until it reaches the end of the array. Even though there may be empty spaces at the front, the queue appears full when rear reaches the last index. This condition is called **false overflow** or **queue overflow**.

Example of Linear Queue Problem:
Array size = 5
• Insert A, B, C, D, E → rear = 4 (full)
• Delete A, B → front = 2 (positions 0, 1 are vacant)
• Try to insert F → rear cannot move beyond 4, even though 2 spaces are empty!

How Circular Queue Overcomes This Limitation:
In a circular queue, when the rear reaches the end of the array, it wraps around to the beginning using modulo arithmetic:
• **Enqueue:** rear = (rear + 1) % SIZE
• **Dequeue:** front = (front + 1) % SIZE
This creates a circular arrangement where vacant positions at the front can be reused by new elements coming in at the rear. The queue is considered full when (rear + 1) % SIZE == front.

Operations in Circular Queue:
// Insertion if ((rear + 1) % SIZE == front): print("Queue is full") else: rear = (rear + 1) % SIZE queue[rear] = element // Deletion if (front == rear): print("Queue is empty") else: front = (front + 1) % SIZE element = queue[front] **Advantages of Circular Queue:**
• Eliminates false overflow problem
• Efficient memory utilization — all array positions can be used
• No need to shift elements after dequeue
• Both enqueue and dequeue operations take O(1) time
• Ideal for CPU scheduling, memory management, and traffic systems

7. What is singly linked list? Write algorithm to add node at beginning and end.

[1+2+2]

Answer: Singly Linked List:
A singly linked list is a linear data structure consisting of a sequence of nodes where each node contains two parts: (1) **data** — the value stored in the node, and (2) **next pointer** — a reference to the next node in the sequence. The last node's next pointer is NULL, indicating the end of the list. The list is accessed through a head pointer that points to the first node.

Structure of a Node:
struct Node { int data; struct Node* next; }; **Algorithm to Add Node at the Beginning:**
Algorithm insertAtBeginning(head, value): 1. Create a new node newNode 2. Set newNode->data = value 3. Set newNode->next = head 4. Set head = newNode 5. Return head **C Implementation:**
struct Node* insertAtBeginning(struct Node* head, int value) { struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)); newNode->data = value; newNode->next = head; head = newNode; return head; } **Example:** If list is 10 → 20 → 30 and we insert 5 at beginning:
New list: 5 → 10 → 20 → 30

Algorithm to Add Node at the End:
Algorithm insertAtEnd(head, value): 1. Create a new node newNode 2. Set newNode->data = value 3. Set newNode->next = NULL 4. If head == NULL: head = newNode Return head 5. Set temp = head 6. While temp->next != NULL: temp = temp->next 7. Set temp->next = newNode 8. Return head **C Implementation:**
struct Node* insertAtEnd(struct Node* head, int value) { struct Node* newNode = (struct Node*)malloc(sizeof(struct Node)); struct Node* temp = head; newNode->data = value; newNode->next = NULL; if (head == NULL) { head = newNode; return head; } while (temp->next != NULL) { temp = temp->next; } temp->next = newNode; return head; } **Example:** If list is 10 → 20 → 30 and we insert 40 at end:
New list: 10 → 20 → 30 → 40

Time Complexity:
• Insert at beginning: O(1)
• Insert at end: O(n) — requires traversal to the last node

8. Define AVL tree. Construct AVL tree from data set: 4, 6, 12, 9, 5, 2, 13, 8, 3, 7, 11.

[1+4]

Answer: AVL Tree:

An AVL tree (named after its inventors Adelson-Velsky and Landis) is a self-balancing binary search tree where the difference between the heights of the left and right subtrees (balance factor) of every node is at most 1. The balance factor of a node is calculated as: height(left subtree) - height(right subtree). Valid balance factors are -1, 0, or +1. If the balance factor becomes less than -1 or greater than +1 after insertion or deletion, rotations are performed to restore balance.

Balance Factor (BF) = Height of Left Subtree - Height of Right Subtree

- BF = +2 and left child's BF ≥ 0 → Right Rotation (LL)
- BF = +2 and left child's BF < 0 → Left-Right Rotation (LR)
- BF = -2 and right child's BF ≤ 0 → Left Rotation (RR)
- BF = -2 and right child's BF > 0 → Right-Left Rotation (RL)

AVL Tree Construction Step by Step:

Data set: 4, 6, 12, 9, 5, 2, 13, 8, 3, 7, 11

Step 1: Insert 4

4 **Step 2:** Insert 6 (right of 4)

4 \ 6 BF(4) = 0-1 = -1 (balanced)

Step 3: Insert 12 (right of 6)

4 \ 6 \ 12 BF(4) = 0-2 = -2 → **Left Rotate at 4 (RR rotation)**

After rotation:

6 / \ 4 12

Step 4: Insert 9 (left of 12)

6 / \ 4 12 / 9 BF(12) = 1-0 = +1, BF(6) = 0-1 = -1 (balanced)

Step 5: Insert 5 (left of 6, right of 4)

6 / \ 4 12 \ / 5 9 BF(4) = 0-1 = -1, BF(6) = 1-1 = 0 (balanced)

Step 6: Insert 2 (left of 4)

6 / \ 4 12 / \ / 2 5 9 All balanced.

Step 7: Insert 13 (right of 12)

6 / \ 4 12 / \ / \ 2 5 9 13 All balanced.

Step 8: Insert 8 (left of 9)

6 / \ 4 12 / \ / \ 2 5 9 13 / 8 BF(9) = 1-0 = +1, BF(12) = 2-1 = +1, BF(6) = 1-2 = -1 (balanced)

Step 9: Insert 3 (right of 2)

6 / \ 4 12 / \ / \ 2 5 9 13 \ / 3 8 BF(2) = 0-1 = -1, BF(4) = 2-1 = +1, BF(6) = 2-2 = 0 (balanced)

Step 10: Insert 7 (left of 8 or as appropriate)

Insert 7: 7 < 12, 7 < 9, 7 > 8 → right of 8

6 / \ 4 12 / \ / \ 2 5 9 13 \ / \ 3 8 / 7 BF(8) = 1-0 = +1, BF(9) = 2-1 = +1, BF(12) = 2-1 = +1, BF(6) = 2-3 = -1 (balanced)

Step 11: Insert 11 (right of 9)
Insert 11: 11 < 12, 11 > 9, 11 > 8 → check: 11 > 9, goes to right of 9
6 / \ 4 12 / \ / \ 2 5 9 13 \ / \ 3 8 11 / 7 BF(9) = 2-1 = +1, BF(12) = 2-1 = +1. All nodes balanced.

Final AVL Tree:
6 / \ 4 12 / \ / \ 2 5 9 13 \ / \ 3 8 11 / 7 All balance factors are within {-1, 0, +1}. The tree is balanced.

Group C

Attempt any TWO questions. [2×10=20]

9. What is stack? List applications. Write algorithm for PUSH and POP operations.

[2+3+5]

Answer: Stack:
A stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle. The element inserted last is the first one to be removed. It can be visualized as a pile of plates where you can only add or remove plates from the top. The primary operations on a stack are PUSH (insertion) and POP (deletion), both performed at one end called the **TOP** of the stack.

Key Operations:
• **PUSH:** Adds an element to the top of the stack
• **POP:** Removes and returns the top element from the stack
• **PEEK/TOP:** Returns the top element without removing it
• **isEmpty:** Checks if the stack is empty
• **isFull:** Checks if the stack is full (in array implementation)

Applications of Stack:
1. **Expression Evaluation and Conversion:** Converting infix to postfix/prefix and evaluating postfix expressions
2. **Function Call Management:** Call stack in programming languages manages function calls, local variables, and return addresses
3. **Recursion:** Recursive function calls use the system stack to store activation records
4. **Backtracking Algorithms:** Used in maze solving, puzzle solving, and game playing to remember previous states
5. **Undo/Redo Operations:** Text editors and software applications use stacks to implement undo functionality
6. **Browser History:** Back and forward navigation in web browsers
7. **Memory Management:** Runtime memory allocation uses call stacks
8. **Parenthesis Checking:** Validating balanced parentheses in compilers and expression parsers
9. **Tower of Hanoi:** Classic problem solved using recursion/stack
10. **Depth First Search (DFS):** Graph traversal algorithm uses stack

Algorithm for PUSH Operation (Array-based):
Algorithm PUSH(stack, top, maxSize, value): 1. If top == maxSize - 1: Print "Stack Overflow" Return 2. top = top + 1 3. stack[top] = value 4. Print "Element pushed successfully" 5. Return **C Implementation of PUSH:**
#define MAX 100 int stack[MAX]; int top = -1; void push(int value) { if (top == MAX - 1) { printf("Stack Overflow!\n"); return; } top++; stack[top] = value; printf("Pushed: %d\n", value); } **Algorithm for POP Operation (Array-based):**
Algorithm POP(stack, top): 1. If top == -1: Print "Stack Underflow" Return -1 // or error code 2. value = stack[top] 3. top = top - 1 4. Return value **C Implementation of POP:**
int pop() { int value; if (top == -1) { printf("Stack Underflow!\n"); return -1; } value = stack[top]; top--; printf("Popped: %d\n", value); return value; } **Time Complexity:** PUSH = O(1), POP = O(1)
Space Complexity: O(n) where n is the maximum size of the stack

10. What is heap? Explain quick sort with Big-O notation (best, average, worst) and trace to sort: 8, 10, 5, 12, 14, 5, 7, 13.

[2+2+6]

Answer: Heap:
A heap is a specialized tree-based data structure that satisfies the **heap property**. It is a complete binary tree where each node's value follows a specific ordering relative to its children:
• **Max Heap:** The value of each node is greater than or equal to the values of its children. The root contains the maximum element.
• **Min Heap:** The value of each node is less than or equal to the values of its children. The root contains the minimum element.
Properties of Heap:
• It is a complete binary tree (all levels are fully filled except possibly the last, which is filled from left to right)
• Usually implemented using an array where for node at index i:
- Left child at index 2i + 1
- Right child at index 2i + 2
- Parent at index (i - 1) / 2
• Heaps are used in heap sort, priority queues, and scheduling algorithms

Quick Sort:
Quick sort is a divide-and-conquer sorting algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

Algorithm:
Algorithm QuickSort(A, low, high): 1. If low < high: a. pivotIndex = Partition(A, low, high) b. QuickSort(A, low, pivotIndex - 1) c. QuickSort(A, pivotIndex + 1, high) Algorithm Partition(A, low, high): 1. pivot = A[high] 2. i = low - 1 3. For j = low to high - 1: If A[j] <= pivot: i = i + 1 Swap A[i] and A[j] 4. Swap A[i + 1] and A[high] 5. Return i + 1 **Time Complexity of Quick Sort:**
CaseComplexityExplanation Best CaseO(n log n)Occurs when pivot divides array into two equal halves at each step Average CaseO(n log n)Expected performance with random input data Worst CaseO(n^2)Occurs when pivot is always smallest or largest (already sorted/reverse sorted array) **Space Complexity:** O(log n) for recursion stack in best/average case, O(n) in worst case.

Trace to Sort: 8, 10, 5, 12, 14, 5, 7, 13
First Call: QuickSort(A, 0, 7)
Array: [8, 10, 5, 12, 14, 5, 7, 13], pivot = 13 (last element)
Partition process:
• j=0, A[0]=8 ≤ 13 → i=0, swap A[0] with A[0] → [8, 10, 5, 12, 14, 5, 7, 13]
• j=1, A[1]=10 ≤ 13 → i=1, swap A[1] with A[1] → [8, 10, 5, 12, 14, 5, 7, 13]
• j=2, A[2]=5 ≤ 13 → i=2, swap A[2] with A[2] → [8, 10, 5, 12, 14, 5, 7, 13]
• j=3, A[3]=12 ≤ 13 → i=3, swap A[3] with A[3] → [8, 10, 5, 12, 14, 5, 7, 13]
• j=4, A[4]=14 > 13 → no swap
• j=5, A[5]=5 ≤ 13 → i=4, swap A[4] and A[5] → [8, 10, 5, 12, 5, 14, 7, 13]
• j=6, A[6]=7 ≤ 13 → i=5, swap A[5] and A[6] → [8, 10, 5, 12, 5, 7, 14, 13]
Swap A[i+1]=A[6] with A[7]: → [8, 10, 5, 12, 5, 7, 13, 14]
Pivot index = 6

Left Sub-array: QuickSort(A, 0, 5)
Array: [8, 10, 5, 12, 5, 7], pivot = 7

• j=0, 8 > 7 → no swap
• j=1, 10 > 7 → no swap
• j=2, 5 ≤ 7 → i=0, swap A[0] and A[2] → [5, 10, 8, 12, 5, 7]
• j=3, 12 > 7 → no swap
• j=4, 5 ≤ 7 → i=1, swap A[1] and A[4] → [5, 5, 8, 12, 10, 7]
• j=5, A[5]=7 is pivot position, end loop
Swap A[i+1]=A[2] with A[5]: → [5, 5, 7, 12, 10, 8]
Pivot index = 2

Left of 7: QuickSort(A, 0, 1)
Array: [5, 5], pivot = 5
• j=0, 5 ≤ 5 → i=0, swap A[0] with A[0] → [5, 5]
Swap A[1] with A[1]: → [5, 5] (already sorted)

Right of 7: QuickSort(A, 3, 5)
Array: [12, 10, 8], pivot = 8
• j=3, 12 > 8 → no swap
• j=4, 10 > 8 → no swap
Swap A[3] with A[5]: → [8, 10, 12]
Pivot index = 3

Right of 8: QuickSort(A, 4, 5)
Array: [10, 12], pivot = 12
• j=4, 10 ≤ 12 → i=4, swap A[4] with A[4] → [10, 12]
Swap A[5] with A[5] → [10, 12] (already sorted)

Right of original pivot (13): QuickSort(A, 7, 7)
Single element [14], already sorted.

Final Sorted Array: [5, 5, 7, 8, 10, 12, 13, 14]

11. Define graph and tree. Explain BFS and DFS with examples.

[2+2+6]

Answer: Graph:
A graph is a non-linear data structure consisting of a finite set of **vertices (nodes)** and a set of **edges** that connect pairs of vertices. Formally, a graph G is defined as G = (V, E) where V is the set of vertices and E is the set of edges. Graphs can be **directed** (edges have direction) or **undirected** (edges have no direction). Edges may have **weights** representing costs, distances, or capacities.
Types of Graphs:
• **Undirected Graph:** Edges have no direction; if (u, v) exists, (v, u) also exists
• **Directed Graph (Digraph):** Edges have direction; (u, v) ≠ (v, u)
• **Weighted Graph:** Each edge has an associated weight/cost
• **Unweighted Graph:** All edges are treated equally
• **Cyclic Graph:** Contains at least one cycle
• **Acyclic Graph:** Contains no cycles (e.g., DAG — Directed Acyclic Graph)
• **Complete Graph:** Every pair of distinct vertices is connected by an edge
• **Connected Graph:** There is a path between every pair of vertices

Tree:
A tree is a special type of graph that is **connected, acyclic, and undirected**. It consists of nodes connected by edges with the following properties:
• There is exactly one path between any two nodes
• A tree with n nodes has exactly n-1 edges
• One node is designated as the **root**
• Nodes with no children are called **leaves**
• Nodes with children are called **internal nodes**
Types of Trees:
• **Binary Tree:** Each node has at most 2 children
• **Binary Search Tree (BST):** Left child < parent < right child
• **AVL Tree:** Self-balancing BST
• **B-Tree/B+ Tree:** Multi-way search trees used in databases
• **Heap:** Complete binary tree with heap property

Difference between Graph and Tree:
Graph**Tree** May contain cyclesCannot contain cycles Not necessarily connectedAlways connected No root nodeHas a root node Number of edges not fixedn nodes have exactly n-1 edges No parent-child hierarchyParent-child hierarchy exists More general structureSpecial case of graph
Breadth First Search (BFS):
BFS is a graph traversal algorithm that explores vertices level by level. It starts from a source vertex, visits all its adjacent vertices, then visits their adjacent vertices, and so on. BFS uses a **queue** data structure.
BFS Algorithm:
Algorithm BFS(graph, start): 1. Create a queue Q 2. Mark start as visited and enqueue(Q, start) 3. While Q is not empty: a. vertex = dequeue(Q) b. Process vertex c. For each neighbor of vertex: If neighbor is not visited: Mark neighbor as visited enqueue(Q, neighbor) **Example of BFS:**
Consider the following undirected graph:
0 / \ 1---2 | | 3---4 Starting from vertex 0:
• Visit 0, enqueue 0 → Queue: [0]
• Dequeue 0, enqueue neighbors 1, 2 → Queue: [1, 2]
• Dequeue 1, enqueue neighbors 3 (2 already visited) → Queue: [2, 3]
• Dequeue 2, enqueue neighbors 4 (1 already visited) → Queue: [3, 4]

• Dequeue 3, no new neighbors → Queue: [4]
• Dequeue 4, no new neighbors → Queue: []
BFS Order: 0, 1, 2, 3, 4

Depth First Search (DFS):
DFS is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It goes deep into the graph before exploring neighbors at the same level. DFS uses a **stack** (explicit or implicit via recursion).
DFS Algorithm (Recursive):
Algorithm DFS(graph, vertex): 1. Mark vertex as visited 2. Process vertex 3. For each neighbor of vertex: If neighbor is not visited: DFS(graph, neighbor) **DFS Algorithm (Iterative using Stack):**
Algorithm DFS(graph, start): 1. Create a stack S 2. Mark start as visited and push(S, start) 3. While S is not empty: a. vertex = pop(S) b. Process vertex c. For each neighbor of vertex: If neighbor is not visited: Mark neighbor as visited push(S, neighbor) **Example of DFS (same graph as above):**
Starting from vertex 0:
• Push 0, visit 0 → Stack: [0]
• Pop 0, push neighbors 2, 1 → Stack: [2, 1]
• Pop 1, push neighbors 3 (2 already visited, 0 visited) → Stack: [2, 3]
• Pop 3, push neighbors 4 (1 visited) → Stack: [2, 4]
• Pop 4, no new neighbors (2, 3 visited) → Stack: [2]
• Pop 2, no new neighbors (0, 1, 4 visited) → Stack: []
DFS Order: 0, 1, 3, 4, 2

Comparison of BFS and DFS:
AspectBFSDFS Data StructureQueueStack ExplorationLevel by level (breadth-wise)Depth-wise, then backtrack Shortest PathFinds shortest path in unweighted graphsDoes not guarantee shortest path Memory UsageHigher (stores all nodes at current level)Lower (stores only current path) CompletenessComplete (finds solution if it exists)Complete for finite graphs ApplicationsShortest path, level-order traversal, social networksTopological sort, cycle detection, maze solving, backtracking Time ComplexityO(V + E)O(V + E) Space ComplexityO(V)O(V)

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.